Automation Framework
---
we organize automation project files/foldes in structured manner.

Objectives
---
1) Re-usubility
2) Maintainability
3) Readability

Types of frameworks
---
1) Built-in
Ex: testng, junit, cucumber, robot etc..

2) customized(user defined)
modular framework, data driven, keywork driven, hybrid driven etc..

Phases/stages
---
1) Analysing AUT

- Number of pages
- what are all elements / how/ type
- what to automate / what we cannot automate

2) choose test cases for automation

100 Test cases -----90 automatable 10 cannot automate
100% automation

1)sanity test cases - P1
2) data driven test cases/re-tests - P2
3) Regression test cases -P3
4) Any other cases - P4

3) Design & Developement of framework

4) Execution - local, remotely

5) Maintenace

E-commerce domain
---

1) Frontent operations (customers/users)

- register an account
- login
- search for the product
- add/edit/delete products from cart
- order products
- reviews
etc....

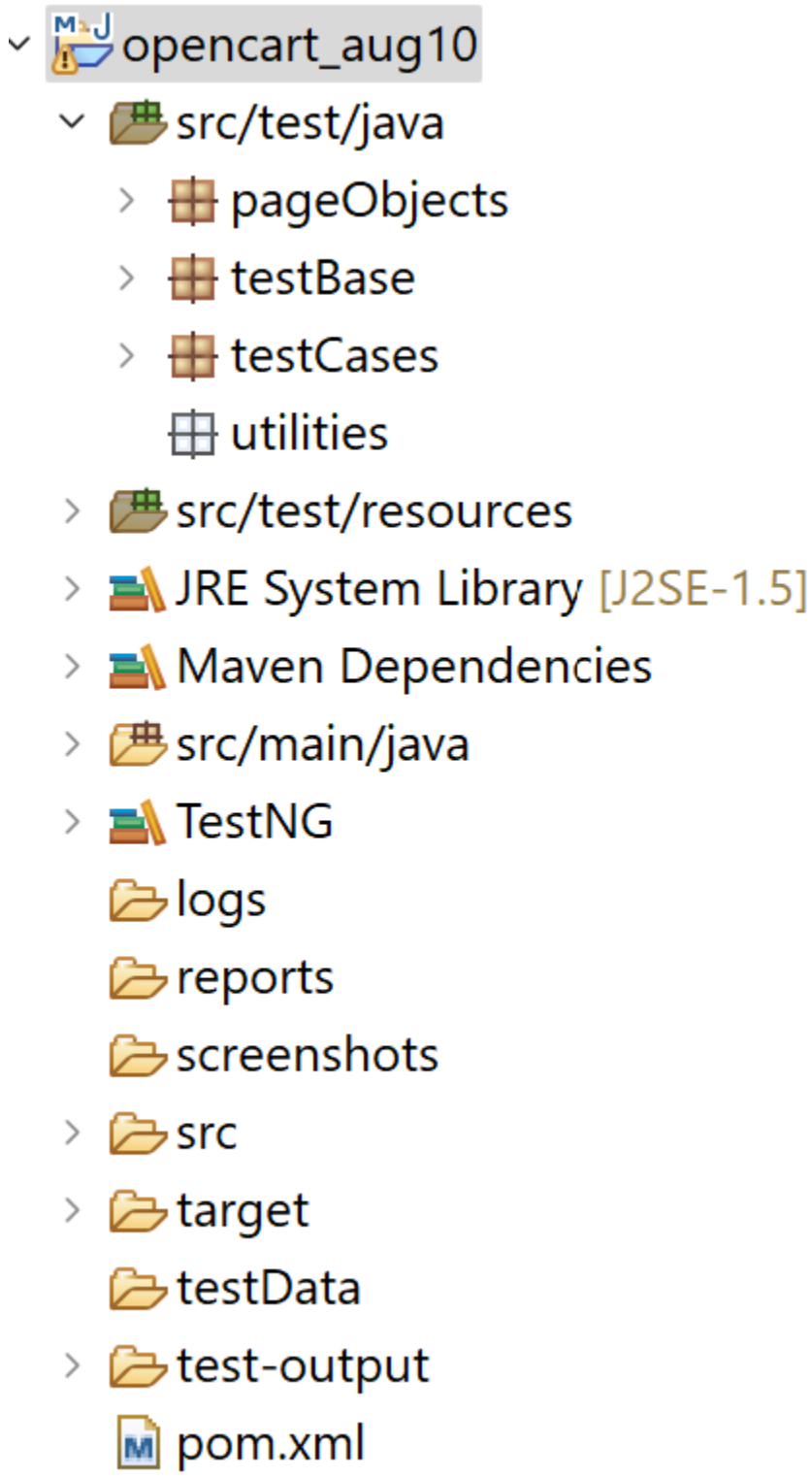
2) Backend operations (admins/backend teams)
- products
- customers
- orders
- store

etc..

opencart app:
---
https://tutorialsninja.com/demo/

"There are several types of automation frameworks, each with its own structure and use case:

1. **Linear Framework** – This is a simple record-and-playback approach. It's quick to set up but not scalable.
2. **Modular Framework** – Test cases are broken into small reusable modules. It improves maintainability and reduces duplication.
3. **Library Architecture** – Common functions are stored in libraries and called when needed. It's useful for repetitive actions like login or data export.
4. **Data-Driven Framework** – Test logic is separated from test data. We use external files like Excel or CSV to run the same test with multiple inputs.

5. **Keyword-Driven Framework** – Actions are defined by keywords like “Click” or “EnterText.” It allows non-technical users to write tests using predefined keywords.
6. **Hybrid Framework** – This combines multiple frameworks, like modular + data-driven + keyword-driven. It’s flexible and widely used in real-world projects.
7. 

8. **BDD (Behavior-Driven Development)** – Tests are written in plain English using tools like Cucumber. It helps bridge communication between QA, developers, and business teams.

Day-49
---

http://localhost/opencart/upload/

1) Test case: Account Registration
1.1: Create BasePage under "pageObjects" which includes only constructor. This will be invoked by every Page Object Class constructor (Re-usability).
1.2: Create Page Object Classes for HomePage, RegistrationPage under pageObjects package. (These classes extends from BasePage).
1.3: Create AccountRegistrationTest under "testCases"
1.4: Create BaseClass under testBase package and copy re-usable methods.
1.5: Create re-usable methods to generate random numbers and strings in BaseClass.

Add required dependencies in pom.xml (Please check links below)
---
https://mvnrepository.com/artifact/org.seleniumhq.selenium/selenium-java
https://mvnrepository.com/artifact/org.apache.poi/poi
https://mvnrepository.com/artifact/org.apache.poi/poi-ooxml
https://mvnrepository.com/artifact/org.apache.logging.log4j/log4j-core
https://mvnrepository.com/artifact/org.apache.logging.log4j/log4j-api
https://mvnrepository.com/artifact/commons-io/commons-io
https://mvnrepository.com/artifact/org.apache.commons/commons-lang3
https://mvnrepository.com/artifact/org.testng/testng
https://mvnrepository.com/artifact/com.aventstack/extentreports

Day-50

Logging - record all the events in the form of text.

Log Levels - All < Trace < Debug < Info < Warn < Error < Fatal < Off

Appenders - where to generate logs (Console/File)

Loggers - what type of logs generate (All < Trace < Debug < Info < Warn < Error < Fatal < Off)

```
import org.apache.logging.log4j.LogManager;  //Log4j
import org.apache.logging.log4j.Logger;  //Log4j
```

Q1. What is logging in automation testing? A: Logging means recording events or actions during test execution. It helps track what happened, when, and where — useful for debugging and reporting.

Q2. What are log levels in Log4j? A: Log levels define the importance of messages:

- All – logs everything
- Trace – very detailed logs
- Debug – for debugging info
- Info – general information
- Warn – warning messages
- Error – error messages
- Fatal – severe errors
- Off – disables logging

Q3. What are appenders and loggers in Log4j? A:

- **Appenders** decide *where* logs go (e.g., console, file).
- **Loggers** decide *what type* of logs to generate (based on log level).

Q4. How do you implement Log4j in your framework? A: I use Log4j to capture test steps, warnings, and errors. I configure it via log4j2.xml and use Logger in my classes to log messages at different levels.

🧪 **Sample Code for Log4j Integration**

java

```
import org.apache.logging.log4j.LogManager;

import org.apache.logging.log4j.Logger;
```

```
public class EmiCalculatorTest {


   // Create logger instance

   private static final Logger log = LogManager.getLogger(EmiCalculatorTest.class);
this.getClass()


   public static void main(String[] args) {

      log.info("Test started: EMI Calculator");

      log.debug("Opening browser and navigating to URL");


      try {

         // Simulate test steps

         log.trace("Setting loan amount slider");

         log.info("Loan amount set to ₹5,00,000");

         log.warn("Interest rate seems unusually high");

         log.error("EMI calculation failed due to missing element");

      } catch (Exception e) {

         log.fatal("Test crashed: " + e.getMessage());

      }


      log.info("Test completed");

   }

}
```

🛠 Sample log4j2.xml Configuration (Basic)

xml

```
<?xml version="1.0" encoding="UTF-8"?>

<Configuration status="WARN">

  <Appenders>
```

```
    <Console name="Console" target="SYSTEM_OUT">
      <PatternLayout pattern="%d [%t] %-5level: %msg%n%throwable"/>
    </Console>
    <File name="FileLogger" fileName="logs/testlog.log">
      <PatternLayout pattern="%d [%t] %-5level: %msg%n"/>
    </File>
  </Appenders>

  <Loggers>
    <Root level="debug">
      <AppenderRef ref="Console"/>
      <AppenderRef ref="FileLogger"/>
    </Root>
  </Loggers>
</Configuration>
```

1. What is Selenium Grid?

Answer: Selenium Grid is a tool that allows you to run tests on different machines and browsers in parallel. It helps in distributed test execution, reducing test execution time and supporting cross-browser and cross-platform testing.

2. What are the main components of Selenium Grid?

Answer:

- **Hub:** Central server that controls test execution. It receives test requests and routes them to the right Node.
- **Node:** Machines (VMs or physical) where actual test execution happens. Each Node can run multiple browser instances.

3. How do you set up Selenium Grid?

Answer:

1. Download the Selenium Server JAR file.
2. Start the Hub:

bash

java -jar selenium-server-<version>.jar hub

3. Start a Node and register it to the Hub:

bash

java -jar selenium-server-<version>.jar node --detect-drivers true --hub http://localhost:4444

4. How do you configure a Node with specific browser capabilities?

Answer: You can use a JSON config file to define browser types, versions, and max instances. Then start the Node like this:

bash

java -jar selenium-server-<version>.jar node --config nodeConfig.json

5. How do you run tests on Selenium Grid using Java?

Answer: Use RemoteWebDriver with DesiredCapabilities and the Hub URL.

java

```
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.remote.RemoteWebDriver;
import org.openqa.selenium.remote.DesiredCapabilities;
import java.net.URL;

public class GridTest {
   public static void main(String[] args) throws Exception {
      DesiredCapabilities cap = new DesiredCapabilities();
      cap.setBrowserName("chrome");

      WebDriver driver = new RemoteWebDriver(new
URL("http://localhost:4444/wd/hub"), cap);
      driver.get("https://emicalculator.net");
```

```
        System.out.println("Title: " + driver.getTitle());
        driver.quit();
    }
}
```

6. What are the benefits of using Selenium Grid?

Answer:

- Parallel execution → faster test cycles
- Cross-browser and cross-platform testing
- Scalable and flexible test infrastructure
- Centralized control via Hub

7. What's the difference between Selenium Grid 3 and Grid 4?

Answer:

- **Grid 4** is fully distributed (Hub, Session Map, Distributor, Router)
- Supports **Docker**, **Kubernetes**, and **self-healing nodes**
- Uses **new CLI** and **YAML/JSON** configs
- Better **observability** with built-in dashboards

8. How do you integrate Selenium Grid with TestNG?

Answer: Use @Parameters in testng.xml to pass browser/platform info, and initialize RemoteWebDriver in your @BeforeMethod.

9. Can you run Selenium Grid on the cloud?

Answer: Yes. Tools like **BrowserStack**, **Sauce Labs**, and **LambdaTest** offer cloud-based Selenium Grid with pre-configured browsers and OS combinations.

10. How do you troubleshoot Grid issues?

Answer:

- Check Hub and Node logs
- Ensure Node is registered (check Grid console)
- Validate browser drivers and versions
- Confirm network/firewall settings

```
package testBase;

import java.io.File;
import java.io.FileReader;
import java.io.IOException;
import java.net.URL;
import java.text.SimpleDateFormat;
import java.time.Duration;
import java.util.Date;
import java.util.Properties;

import org.apache.commons.lang3.RandomStringUtils;
import org.openqa.selenium.OutputType;
import org.openqa.selenium.Platform;
import org.openqa.selenium.TakesScreenshot;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.edge.EdgeDriver;
import org.openqa.selenium.firefox.FirefoxDriver;
import org.openqa.selenium.remote.DesiredCapabilities;
import org.openqa.selenium.remote.RemoteWebDriver;
import org.testng.annotations.AfterClass;
import org.testng.annotations.BeforeClass;
import org.testng.annotations.Parameters;

import org.apache.logging.log4j.LogManager;  //Log4j
import org.apache.logging.log4j.Logger;  //Log4j

public class BaseClass {

public static WebDriver driver;
public Logger logger;  //Log4j
public Properties p;

  @BeforeClass(groups= {"Sanity","Regression","Master"})
  @Parameters({"os","browser"})
  public void setup(String os, String br) throws IOException
  {
    //Loading config.properties file
    FileReader file=new FileReader("./src//test//resources//config.properties");
    p=new Properties();
    p.load(file);

    logger=LogManager.getLogger(this.getClass());  //lOG4J2
```

```
    if(p.getProperty("execution_env").equalsIgnoreCase("remote"))
    {
      DesiredCapabilities capabilities=new DesiredCapabilities();

      //os
      if(os.equalsIgnoreCase("windows"))
      {
        capabilities.setPlatform(Platform.WIN11);
      }
      else if (os.equalsIgnoreCase("mac"))
      {
        capabilities.setPlatform(Platform.MAC);
      }
      else
      {
        System.out.println("No matching os");
        return;
      }

      //browser
      switch(br.toLowerCase())
      {
      case "chrome": capabilities.setBrowserName("chrome"); break;
      case "edge": capabilities.setBrowserName("MicrosoftEdge"); break;
      default: System.out.println("No matching browser"); return;
      }

      driver=new RemoteWebDriver(new
URL("http://localhost:4444/wd/hub"),capabilities);
    }


    if(p.getProperty("execution_env").equalsIgnoreCase("local"))
    {

      switch(br.toLowerCase())
      {
      case "chrome" : driver=new ChromeDriver(); break;
      case "edge" : driver=new EdgeDriver(); break;
      case "firefox": driver=new FirefoxDriver(); break;
      default : System.out.println("Invalid browser name.."); return;
      }
    }


    driver.manage().deleteAllCookies();
    driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
```

```
    driver.get(p.getProperty("appURL2")); // reading url from properties file.
    driver.manage().window().maximize();
  }

  @AfterClass(groups= {"Sanity","Regression","Master"})
  public void tearDown()
  {
    driver.quit();
  }

  public String randomeString()
  {
    String generatedstring=RandomStringUtils.randomAlphabetic(5);
    return generatedstring;
  }

  public String randomeNumber()
  {
    String generatednumber=RandomStringUtils.randomNumeric(10);
    return generatednumber;
  }

  public String randomeAlphaNumberic()
  {
    String generatedstring=RandomStringUtils.randomAlphabetic(3);
    String generatednumber=RandomStringUtils.randomNumeric(3);
    return (generatedstring+"@"+generatednumber);
  }

  public String captureScreen(String tname) throws IOException {

    String timeStamp = new SimpleDateFormat("yyyyMMddhhmmss").format(new
Date());

    TakesScreenshot takesScreenshot = (TakesScreenshot) driver;
    File sourceFile = takesScreenshot.getScreenshotAs(OutputType.FILE);

    String targetFilePath=System.getProperty("user.dir")+"\\screenshots\\" + tname +
"_" + timeStamp + ".png";
    File targetFile=new File(targetFilePath);

    sourceFile.renameTo(targetFile);

    return targetFilePath;

  }
```

```
}
```

Standalone Setup (Single machine):

1. Download selenium-server-4.20.0.jar and place it somewhere.
2. Run below command to start Selenium Grid

java -jar selenium-server-4.20.0.jar *standalone*

3. URL to see sessions: http://localhost:4444/

Hub & Node Setup (Multiple machines):

1. Download selenium-server-4.20.0.jar and place it somewhere in both (hub & node) the machines.
2. Run below command to make machine as hub

java -jar selenium-server-4.20.0.jar hub

3. Run below command to make machine as node

java -jar selenium-server-4.20.0.jar node --hub http://<hub-ip>:4444

Example:

java -jar '/home/pavan/Desktop/selenium-server-4.20.0.jar' node --hub http://192.168.1.6:4444 --selenium-manager true

4. URL to see sessions: http://localhost:4444/

Standalone Setup (Single machine):
4. Download selenium-server-4.20.0.jar and place it somewhere.
5. Run below command to start Selenium Grid

java -jar selenium-server-4.20.0.jar *standalone*

6. URL to see sessions: http://localhost:4444/

Hub & Node Setup (Multiple machines):

5. Download selenium-server-4.20.0.jar and place it somewhere in both (hub & node) the machines.
6. Run below command to make machine as hub

java -jar selenium-server-4.20.0.jar hub

7. Run below command to make machine as node

java -jar selenium-server-4.20.0.jar node --hub http://<hub-ip>:4444

Example:

java -jar '/home/pavan/Desktop/selenium-server-4.20.0.jar' node --hub http://192.168.1.6:4444 --selenium-manager true

8. URL to see sessions: http://localhost:4444/

Day-55

Run Tests using Maven pom.xml, Command Prompt & run.bat file.

---

pom.xml

---

dependencies --- download required dependency jar for project

plugins ---> to compile and run the project

1) maven compiler plugin
2) maven surefire plugin

Install maven OS level

---

https://maven.apache.org/download.cgi

C:\Program Files\apache-maven-3.9.6\bin

** configure the maven path in environment variable.

open command prompt --> mvn -version

Configure Maven path in Mac OS

In terminal, type echo $SHELL

In terminal, type vi ~/.zshrc

Press i for insert and type export MAVEN_HOME=$(/usr/libexec/maven_home)

Press esc :wq

In the terminal:

```
echo $MAVEN_HOME
mvn -version
```

Git & GitHub

Project location:

C:\Automation\myworkspaces\seleniumwebdriver\OpencartV121

```
cd C:\Automation\myworkspaces\seleniumwebdriver\OpencartV121
mvn test
```

Pre-requisites
---
1) Git installation
2) create an account with GitHub & create remote repository then capture url
   Account creation: https://github.com/signup?source=login
   repository url: https://github.com/pavanoltraining/OpencartV121.git

Git & GitHub workflow

Round1
---
1) create a new local repository (one time)

```
git init
```

2) provided user info to git repo. (one time)

```
git config --global user.name "your name"
git config --global user.email "your email"
```

3) Adding files/folders to staging/indexing

```
git add -A       -->add all the files and folders to staging
git add filename
git add *.java
```

```
   git add foldername

4) Commit the code into Local(git) repository
        git commit -m "commit message"

5) connect local repository with remote rep.(One time)
        git remote add origin "https://github.com/pavanoltraining/OpencartV121.git"


6) push the code into remote repository.
        git push origin master

token: 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   (**Create your own Token)



Round2
------
1) git add -A
2) git commit -m "commit message"
3) git push origin master

Pull the files from remote repository
---------------
git pull "https://github.com/pavanoltraining/OpencartV121.git"



clone new repository to local system
------------------------
git clone "https://github.com/pavanoltraining/testrepo.git"
```

- **MAVEN**

Q1. What is Maven and why do we use it in automation projects? A: Maven is a build automation tool used primarily for Java projects. In automation, we use Maven to manage dependencies (like Selenium, TestNG), compile code, and run test cases using plugins like Surefire.

Q2. What is pom.xml **in Maven? A:** pom.xml (Project Object Model) is the heart of a Maven project. It contains:

- **Dependencies**: External libraries like Selenium, TestNG, Apache POI
- **Plugins**: Tools to compile code, run tests, generate reports
- **Build Configurations**: Java version, source directory, etc.

Q3. How do you run your test cases using Maven? A:

1. Navigate to the project directory:

bash

cd C:\Automation\myworkspaces\seleniumwebdriver\OpencartV121

2. Run tests using:

bash

mvn test

Q4. What plugins do you use in pom.xml**? A:**

- **maven-compiler-plugin**: Compiles Java code
- **maven-surefire-plugin**: Executes TestNG/JUnit test cases

📄 Sample pom.xml snippet:

xml

```
<build>
 <plugins>
  <plugin>
   <groupId>org.apache.maven.plugins</groupId>
   <artifactId>maven-compiler-plugin</artifactId>
   <version>3.8.1</version>
   <configuration>
    <source>1.8</source>
    <target>1.8</target>
   </configuration>
  </plugin>

  <plugin>
```

```
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-surefire-plugin</artifactId>
    <version>2.22.2</version>
    <configuration>
     <suiteXmlFiles>
      <suiteXmlFile>testng.xml</suiteXmlFile>
     </suiteXmlFiles>
    </configuration>
   </plugin>
 </plugins>
</build>
```

Q5. How do you configure Maven on Windows and Mac? A:

- **Windows**:
    - Download Maven → Set MAVEN_HOME and add bin to PATH
    - Verify: mvn -version
- **Mac**:

```
bash
echo $SHELL
vi ~/.zshrc
export MAVEN_HOME=$(/usr/libexec/maven_home)
source ~/.zshrc
mvn -version
```

Q6. How do you run tests using a .bat **file? A:** Create a run.bat file with:

```
bat
cd C:\Automation\myworkspaces\seleniumwebdriver\OpencartV121
mvn clean test
pause
```

Double-click to execute tests.

🔹 **GIT & GITHUB**

Q7. What is Git and how is it different from GitHub? A:

- **Git**: Version control system to track code changes locally
- **GitHub**: Cloud platform to host Git repositories and collaborate with teams

Q8. How do you push your automation code to GitHub? A:

bash

git init

git config --global user.name "Komal"

git config --global user.email "komal@example.com"

git add -A

git commit -m "Initial commit"

git remote add origin https://github.com/komal/OpencartV121.git

git push origin master

Q9. How do you pull or clone a project from GitHub? A:

- **Clone**:

bash

git clone https://github.com/komal/testrepo.git

- **Pull**:

bash

git pull https://github.com/komal/OpencartV121.git

Q10. What is the Git workflow you follow in your project? A: 🔁 **Round 1 (Initial Setup)**

1. git init
2. git config --global user.name and user.email
3. git add -A
4. git commit -m "Initial commit"
5. git remote add origin <repo_url>
6. git push origin master

🔁 Round 2 (Daily Updates)

1. git add -A
2. git commit -m "Updated test cases"
3. git push origin master

Q11. What is a Git token and why is it needed? A: A GitHub token replaces your password when pushing code from Git to GitHub (especially after 2021, GitHub removed password-based authentication). You generate it from your GitHub account settings.

🍼 Jenkins Interview Questions & Answers

🔹 Basics & Setup

Q1. What is Jenkins? A: Jenkins is an open-source automation server used for Continuous Integration (CI) and Continuous Delivery (CD). It automates building, testing, and deploying code.

Q2. What are the key features of Jenkins? A:

- Easy plugin integration
- Distributed builds
- Supports multiple languages and tools
- Scheduled or triggered builds
- Real-time feedback and reporting

Q3. How do you install Jenkins? A:

- Download from jenkins.io
- Install via .war file, Docker, or native packages
- Access via http://localhost:8080

Jenkins.pdf

Local project

🛠️ 2. Create a Maven Job in Jenkins

1. **Open Jenkins Dashboard**

2. Click **“New Item”**
3. Enter a name (e.g., EMI_Calculator_Tests)
4. Select **“Maven Project”** → Click **OK**

Build → root pom-→ give whole url of pom.xml

Goal & options-> test

1.In case of github repository add url in github section

3.bat file create freestyle project→build step→ Execute windows batch command→ mention whole url of bat file

Cucumber

Cucumber

TDD - Test Driven Development

Ex: JUnit/TestNG (for Java), NUnit (for .NET), PyTest (for Python), Jasmine (for JavaScript)

BDD - Behavioral Driven Development

Ex: Cucumber (supports multiple languages like Java, Ruby, JavaScript, etc.)
SpecFlow (for .NET)
Behave (for Python)
JBehave (for Java)

team
---
stake holders/customer (non-tech)
product manager (non-tech)
project manager - (non-tech)
tester - tech

developer - tech
scrum master (non-tech)

BDD, or Behavior-Driven Development, emphasizes collaboration between technical and non-technical stakeholders
through the use of a common language to describe the behavior of a system.
It focuses on writing tests in a human-readable format, typically using a "given-when-then" structure, which helps clarify requirements and expectations.

Cucumber
---
1) Feature file
2) step definitions file
3) test runner file

Feature file contains scenarios & Steps.

Gherkin - language contains keywords..

Scenario
Given
When
Then
And

Pre-condition - Given
Actions - When
Validations - Then

Login.feature

```
Feature : User login
        Scenario: Succesful login
                Given the user opens application
                And  the user navigate to login page
                When the user entered valid user name and valid password
                And the user clicked on submit button.
                Then the user should see My account page.
                And the user name should be displayed on my account page
```

Step definition file contains methods which are represent to steps in feature file

testRunner class ---> feature file, steps, report etc...

Cucumber project setup
---
1) Install cucumber eclipse plugin (From eclipse market place)
2) Create Maven project
3) Update pom.xml with required dependencies.

- **Implementation Example**

Login.feature

gherkin

```
Feature: User login


  Scenario: Successful login
    Given the user opens application
    And the user navigates to login page
    When the user enters valid username and password
    And clicks on the submit button
    Then the user should see My Account page
    And the username should be displayed
```

StepDefinition.java

java

```
public class LoginSteps {


    WebDriver driver;


    @Given("the user opens application")
    public void openApplication() {
        driver = new ChromeDriver();
```

```
    driver.get("https://example.com");
}

@And("the user navigates to login page")
public void navigateToLogin() {
    driver.findElement(By.linkText("Login")).click();
}

@When("the user enters valid username and password")
public void enterCredentials() {
    driver.findElement(By.id("username")).sendKeys("komal");
    driver.findElement(By.id("password")).sendKeys("password123");
}

@And("clicks on the submit button")
public void clickSubmit() {
    driver.findElement(By.id("submit")).click();
}

@Then("the user should see My Account page")
public void verifyAccountPage() {
    Assert.assertTrue(driver.getTitle().contains("My Account"));
}

@And("the username should be displayed")
public void verifyUsername() {
    Assert.assertTrue(driver.getPageSource().contains("komal"));
    driver.quit();
```

```
  }
}
```

TestRunner.java

```
java
@RunWith(Cucumber.class)
@CucumberOptions(
 features = "src/test/resources/features",
 glue = "stepDefinitions",
 plugin = {"pretty", "html:target/cucumber-reports"},
 monochrome = true
)
public class TestRunner {}
```

- **Maven Setup**

Q5. What dependencies are required in pom.xml **for Cucumber? A:**

```
xml
<dependencies>
 <dependency>
  <groupId>io.cucumber</groupId>
  <artifactId>cucumber-java</artifactId>
  <version>7.14.0</version>
 </dependency>
 <dependency>
  <groupId>io.cucumber</groupId>
  <artifactId>cucumber-junit</artifactId>
  <version>7.14.0</version>
  <scope>test</scope>
 </dependency>
 <dependency>
```

```
  <groupId>org.seleniumhq.selenium</groupId>
  <artifactId>selenium-java</artifactId>
  <version>4.21.0</version>
 </dependency>
 <dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>4.13.2</version>
  <scope>test</scope>
 </dependency>
</dependencies>
```

🔹 **Advanced Questions**

Q6. How do you handle parameterization in Cucumber? A:

🧩 **1. Using Regular Expressions in Step Definitions**

You can capture dynamic values from feature files using regex groups:

```
java
@Given("^user logs in with username \"([^\"]*)\" and password \"([^\"]*)\"$")
public void login(String username, String password) {
  // Use the parameters in your test logic
}
```

- \" → Matches a literal double quote ("). The backslash escapes it.
- ([^\"]*) → Capturing group:
  - [^\"] → Match any character **except** a double quote.
  - * → Match **zero or more** of those characters.
- \" → Matches the closing double quote.

✅ **Feature File Example:**

gherkin

Given user logs in with username "komal123" and password "securePass"

📦 2. Using Cucumber Expressions (Simpler Syntax)

Cucumber also supports a cleaner syntax called *Cucumber Expressions*:

java

```
@Given("user logs in with username {string} and password {string}")
public void login(String username, String password) {
   // cleaner and more readable
}
```

✅ Feature File Example:

gherkin

Given user logs in with username "komal123" and password "securePass"

📊 3. Scenario Outline with Examples Table

For multiple data sets, use Scenario Outline:

gherkin

```
Scenario Outline: Login with multiple credentials
 Given user logs in with username "<username>" and password "<password>"

Examples:
 | username  | password    |
 | komal123  | securePass  |
 | testUser  | test123     |
```

✅ Step Definition:

java

```
@Given("user logs in with username {string} and password {string}")
public void login(String username, String password) {
   // works seamlessly with scenario outline
}
```

Scenario Outline: Verify player ranking based on score

Given the player "<name>" has a score of "<score>"

When the ranking is calculated

Then the player should be in "<rank>" position

Examples:

| name | score | rank |

| Kohli | 850 | Top 3 |

| Smith | 780 | Top 5 |

| Root | 720 | Top 10 |

```
package stepDefinitions;

import io.cucumber.java.en.*;
import org.junit.Assert;
import java.util.HashMap;
import java.util.Map;

public class PlayerRankingSteps {

   private String playerName;
   private int playerScore;
   private String actualRank;

   // Simulated ranking logic (replace with real logic or page object calls)
   private String calculateRank(int score) {
      if (score >= 800) return "Top 3";
      else if (score >= 750) return "Top 5";
      else return "Top 10";
   }
```

```
  @Given("the player {string} has a score of {string}")
  public void the_player_has_a_score(String name, String score) {
    this.playerName = name;
    this.playerScore = Integer.parseInt(score);
    System.out.println("Player: " + name + ", Score: " + score);
  }

  @When("the ranking is calculated")
  public void the_ranking_is_calculated() {
    actualRank = calculateRank(playerScore);
    System.out.println("Calculated Rank for " + playerName + ": " + actualRank);
  }

  @Then("the player should be in {string} position")
  public void the_player_should_be_in_position(String expectedRank) {
    Assert.assertEquals("Ranking mismatch for player: " + playerName,
expectedRank, actualRank);
    System.out.println("Verified Rank: " + expectedRank);
  }
}
```

```
package testRunner;

import org.junit.runner.RunWith;

import io.cucumber.junit.Cucumber;
import io.cucumber.junit.CucumberOptions;

@RunWith(Cucumber.class)
@CucumberOptions(
        //features= {".//Features/"},
        //features= {".//Features/Login.feature"},
        //features= {".//Features/Registration.feature"},

        //features= {".//Features/LoginDDTExcel.feature"},
        features= {".//Features/Login.feature",".//Features/Registration.feature"},
        //features= {"@target/rerun.txt"},
        glue={"stepDefinitions","hooks"},
        plugin= {
                "pretty", "html:reports/myreport.html",
                "com.aventstack.extentreports.cucumber.adapter.ExtentCucumberA
dapter:",
                "rerun:target/rerun.txt",

            },
```

```
            dryRun=false,    // checks mapping between scenario steps and step
definition methods
            monochrome=true,    // to avoid junk characters in output
            publish=true   // to publish report in cucumber server
            //tags="@sanity"  // this will execute scenarios tagged with @sanity
            //tags="@regression"
            //tags="@sanity and @regression" //Scenarios tagged with both @sanity
and @regression
            //tags="@sanity and not @regression" //Scenarios tagged with @sanity but
not tagged with @regression
            //tags="@sanity or @regression" //Scenarios tagged with either @sanity or
@regression
      )
public class TestRunner {

}
```

💉 Cucumber TestRunner – Explained Line by Line

java

```
@CucumberOptions(
 glue = "stepDefinitions",
```

- 🔹 glue: Specifies the package where your step definition classes are located.
- ✅ This tells Cucumber where to find the Java methods that match your Gherkin steps.

java

```
 plugin = {
  "pretty",
  "html:reports/myreport.html",
  "rerun:target/rerun.txt",
  "com.aventstack.extentreports.cucumber.adapter.ExtentCucumberAdapter:"
 },
```

- 🔹 plugin: Defines output formats and reporting tools.
  - "pretty" → Prints readable console output
  - "html:reports/myreport.html" → Generates a basic HTML report
  - "rerun:target/rerun.txt" → Captures failed scenarios for re-execution
  - "ExtentCucumberAdapter" → Generates advanced ExtentReports (requires setup in extent.properties)

java

```
dryRun = false,
```

- 🔹 dryRun: If true, Cucumber checks if every step in the feature file has a matching step definition — without executing the code.
- ✅ Set to false to run actual tests.

java

```
monochrome = true,
```

- 🔹 monochrome: Removes unreadable characters from console output.
- ✅ Set to true for clean, readable logs.

java

```
publish = true,
```

- 🔹 publish: Publishes your test results to the official Cucumber Reports Server.
- ✅ Useful for sharing reports with stakeholders.

🏷️ **Tag Filtering (Commented Examples)**

java

```
// tags = "@sanity"
```

- Runs only scenarios tagged with @sanity

java

```
// tags = "@regression"
```

- Runs only scenarios tagged with @regression

java

```
// tags = "@sanity and @regression"
```

- Runs scenarios tagged with **both** @sanity and @regression

java

```
// tags = "@sanity and not @regression"
```

- Runs scenarios tagged with @sanity but **not** @regression

java

```
// tags = "@sanity or @regression"
```

- Runs scenarios tagged with **either** @sanity or @regression

✅ **Final Class Declaration**

java

```
public class TestRunner {}
```

- This class uses @RunWith(Cucumber.class) (not shown here) to trigger Cucumber execution.
- It connects your feature files, step definitions, and reporting setup.

🔹 **1. Data-Driven Testing with Scenario Outline**

Q: How do you implement data-driven testing in Cucumber? A: I use Scenario Outline with an Examples table to run the same scenario multiple times with different data.

📄 Example:

gherkin

```
Scenario Outline: Login with multiple users
  Given user opens login page
  When user enters "<username>" and "<password>"
  Then user should see dashboard

Examples:
  | username | password  |
  | komal    | pass123   |
```

| admin | admin@123 |

🔹 **2. Data-Driven Testing Using Excel**

Q: How do you read test data from Excel in Cucumber? A: I use Apache POI to read data from Excel. I pass row numbers via Scenario Outline and fetch values dynamically in step definitions.

📜 Feature:

gherkin

Scenario Outline: Login using Excel

Given user opens login page

When user logs in with row <rowNum> from Excel

Then dashboard should be visible

Examples:

| rowNum |

| 1 |

| 2 |

📜 Step:

java

String file = "testdata/LoginData.xlsx";

String username = ExcelUtils.getCellData(file, "Sheet1", rowNum, 0);

String password = ExcelUtils.getCellData(file, "Sheet1", rowNum, 1);

Mobile photo

🔹 **3. DataTable for Multiple Parameters**

Q: How do you pass multiple parameters into one step using DataTable? A: I use Cucumber's DataTable to pass structured data into a single step.

📜 Feature:

gherkin

When user logs in with credentials

| username | password |

| komal | pass123 |

📜 Step:

java

@When("user logs in with credentials")

public void login(DataTable table) {

List<Map<String, String>> data = table.asMaps(String.class, String.class);

String username = data.get(0).get("username");

String password = data.get(0).get("password");

}

🔹 **4. Rerun Failed Scenarios**

Q: How do you rerun only failed scenarios in Cucumber? A: I use the rerun plugin in @CucumberOptions:

java

plugin = {

"pretty",

"rerun:target/rerun.txt"

}

Then create a second runner:

java

@CucumberOptions(

features = "@target/rerun.txt",

glue = "stepDefinitions"

)

🥒 **Cucumber + BDD – Interview Q&A with Implementation**

🔹 **BDD Concepts**

Q1. What is BDD and how is it different from TDD? A:

- **TDD (Test-Driven Development)**: Developers write unit tests before writing code. Tools: JUnit, TestNG.
- **BDD (Behavior-Driven Development)**: Focuses on system behavior using natural language. Encourages collaboration between technical and non-technical stakeholders. Tools: Cucumber, SpecFlow.

Q2. What is Cucumber and why is it used? A: Cucumber is a BDD tool that allows writing test cases in plain English using Gherkin syntax. It bridges the gap between business and technical teams by making test scenarios readable and understandable to all.

3. What basic concepts must one know while working with Cucumber?

To work with Cucumber, you should know these basic concepts:

- **Feature:** A high-level description of a software feature you want to test.
- **Scenario:** A specific situation or test case for the feature.
- **Step:** An action or outcome in the scenario (like "Given," "When," and "Then").
- **Background:** Steps that are common to all scenarios in a feature.
- **Gherkin:** The language used to write Cucumber tests.

4. What is a scenario in Cucumber Testing?

A scenario in Cucumber is a single test case that describes a specific situation or use case for the software. Each scenario includes several steps that describe actions and expected outcomes. For example, a scenario might describe how a user logs into an application and what should happen when they enter the correct username and password.

5. What is Gherkin Language?

Gherkin is the language used to write Cucumber tests. It uses plain English to describe software behaviors in a way that everyone can understand. Gherkin has keywords like "Feature," "Scenario," "Given," "When," and "Then" to organize the test steps. This makes it easy for both technical and non-technical people to read and write tests.

6. What is a Scenario Outline in the Cucumber framework?

A Scenario Outline in Cucumber is used when you need to run the same test scenario multiple times with different sets of data. Instead of writing the same test steps over

and over again, you use a Scenario Outline and provide different data sets in a table. Each row in the table represents a different test case.

7. What is a ‘feature’ in Cucumber?

A ‘feature’ in Cucumber is a high-level description of what a part of your software should do. It is written in a file called a feature file, which has a ".feature" extension. Each feature file contains one or more scenarios that describe different situations of how the feature should work. The feature file uses plain English, so everyone can understand it.

10. What is debugging in Cucumber?

Debugging in Cucumber involves finding and fixing issues in your Cucumber tests or the code they test. Common debugging methods include:

- **Print statements:** Adding print statements in your step definitions to see what's happening at each step.
- **Using a debugger:** Setting breakpoints and running your tests in debug mode to inspect variables and program flow.
- **Dry run:** Running Cucumber in a special mode that checks for missing steps without executing the tests. This helps identify where the problems are.

11. What are the Steps in the context of Cucumber?

In Cucumber, "steps" are individual actions or checks that you perform in a test scenario. Steps are written in plain English and are defined using keywords like "Given," "When," "Then," "And," and "But." Each step is connected to a piece of code called a "step definition" that performs the actual action or check described by the step.

12. What are annotations with respect to Cucumber?

Annotations in Cucumber are special keywords or symbols used in the step definition files to mark specific methods. They help Cucumber understand how to connect the steps written in the feature files with the corresponding code. Common annotations include:

- **@Given:** Marks a method that sets up a precondition.
- **@When:** Marks a method that performs an action.
- **@Then:** Marks a method that checks the outcome.
- **@Before:** Marks a method that runs before each scenario.
- **@After:** Marks a method that runs after each scenario.

13. What are hooks in Cucumber? How can they be used in Cucumber?

Hooks in Cucumber are blocks of code that run at specific points during the test execution cycle. They help manage setup and teardown tasks. You can use hooks to perform actions like opening a browser, setting up test data, or cleaning up after tests.

- **@Before:** Runs before each scenario. Used for setup tasks.
- **@After:** Runs after each scenario. Used for cleanup tasks.

```
@Before
public void setUp() {
   // Code to set up before each scenario
}


@After
public void tearDown() {
   // Code to clean up after each scenario
}
```

14. What are tags in Cucumber, and why are they important?

Tags in Cucumber are labels that you can add to your scenarios or features to organize and control which tests to run. They are written with the "@" symbol followed by a tag name, like @smokeTest or @regression.

Tags are important because they allow you to:

- **Filter tests:** Run only a specific group of tests based on their tags.
- **Organize tests:** Categorize tests into different types (e.g., smoke tests, regression tests).
- **Manage test execution:** Include or exclude tests easily during test runs.

For example:

```
@smokeTest
Scenario: Verify user login
  Given the user is on the login page
  When the user enters valid credentials
  Then the user is redirected to the homepage
```

15. What is a Dry Run in Cucumber?

A Dry Run in Cucumber is a special mode that checks if all the steps defined in the feature files have corresponding step definitions without actually running the tests. This helps identify any missing or unimplemented steps.

You can enable a dry run by setting the dryRun option to true in the Cucumber options. For example:

```
@RunWith(Cucumber.class)
@CucumberOptions(
  features = "src/test/resources/features",
  glue = "stepDefinitions",
  dryRun = true
)
public class TestRunner {
}
```

When you run this, Cucumber will validate the steps and report any missing step definitions.

16. What are Cucumber parameters?

Cucumber parameters allow you to pass dynamic values to your steps. They make your scenarios more flexible and reusable. You define parameters using angle brackets < > in your feature files, and then capture these values in your step definitions.

Example:

Scenario: User logs in
Given the user logs in with username "<username>" and password "<password>"

Step Definition:

```
@Given("the user logs in with username {string} and password {string}")
public void login(String username, String password) {
  // Code to log in with the given username and password
}
```

17. What is meant by a Cucumber profile?

A Cucumber profile is a way to group and run a specific set of tests with certain configurations. You can define profiles in a cucumber.yml file, and each profile can have

different settings like tags to include or exclude, paths to feature files, and other options.

Example cucumber.yml:

default: --tags @regression --glue stepDefinitions --plugin pretty
smoke: --tags @smokeTest --glue stepDefinitions --plugin pretty

To run tests with a specific profile, use the --profile option:

cucumber --profile smoke

18. In which language is Cucumber software written?

Cucumber was originally written in the Ruby programming language. However, it has since been implemented in several other languages, including Java and JavaScript, to support a wider range of developers.

20. What are the two files required to execute a Cucumber test scenario?

To execute a Cucumber test scenario, you need two main files:

A. Feature File (.feature): This file contains the high-level description of the software feature and the test scenarios written in Gherkin language. Example:

Feature: User Login

Scenario: Successful login with valid credentials
Given the user is on the login page
When the user enters valid username and password
Then the user should be redirected to the homepage

B: Step Definition File: This file contains the code that implements the steps defined in the feature file. The step definitions link the Gherkin steps to actual code. Example:

```
@Given("the user is on the login page")
public void userOnLoginPage() {
    // Code to navigate to the login page
}

@When("the user enters valid username and password")
public void userEntersCredentials() {
    // Code to enter username and password
```

```
}

@Then("the user should be redirected to the homepage")
public void userRedirectedToHomepage() {
    // Code to verify redirection to homepage
}
```

21. What are the various keywords used in the Cucumber tool for writing a scenario?

Cucumber uses several keywords in Gherkin to write scenarios. These keywords help to describe the behavior of the software:

- **Feature:** Describes the feature under test.
- **Scenario:** Defines a specific test case with a sequence of steps.
- **Given:** Sets up the initial context or state before an action is taken.
- **When:** Describes the action or event that triggers the test scenario.
- **Then:** Specifies the expected outcome or result of the action.
- **And:** Used to add more steps to Given, When, or Then.
- **But:** Adds a condition to Given, When, or Then.
- **Background:** Defines steps that are common to all scenarios in a feature file.
- **Scenario Outline:** Used to run the same scenario with different sets of data.
- **Examples:** Provides the data sets for a Scenario Outline.

22. What is the use of the Background keyword in Cucumber?

The **Background** keyword in Cucumber is used to define a set of steps that are common to all scenarios in a feature file. It helps avoid repetition by specifying steps that should be run before each scenario. The Background section runs before each scenario, ensuring that the scenarios start with a common context.

Example:

Feature: *User Login*

Background:

Given the user is on the login page

Scenario: *Successful login*

When the user enters valid credentials

Then the user should be redirected to the homepage

Scenario: *Unsuccessful login*

When the user enters invalid credentials

Then the user should see an error message

23. What do you understand by the term step definition in Cucumber?

A **step definition** in Cucumber is the actual code implementation that corresponds to the steps mentioned in the feature file. Each step in the Gherkin scenario is mapped to a method in the step definition file, where the actions described in the step are carried out. Step definitions are written in a programming language like Java, Ruby, or JavaScript.

Example:

Given the user is on the login page

Step Definition in Java:

```
@Given("the user is on the login page")
public void userOnLoginPage() {
   // Code to navigate to the login page
}
```

24. Which programming languages are supported by Cucumber?

Cucumber supports multiple programming languages, making it versatile for different development environments. The supported languages include:

- Ruby (original implementation)
- Java (Cucumber-JVM)
- JavaScript (Cucumber.js)
- .NET (SpecFlow)
- Python (Behave, which is similar to Cucumber)
- PHP (Behat)
- Groovy

- Kotlin
- Scala

25. What are the differences between JBehave and Cucumber?

JBehave and Cucumber are both tools for behavior-driven development (BDD), but they have some differences:

- **Syntax:**
    - JBehave: Uses a story syntax with keywords like GivenStories, Scenario, Given, When, Then, and Examples.
    - Cucumber: Uses Gherkin syntax with keywords like Feature, Scenario, Given, When, Then, And, But, and Examples.
- **Language:**
    - JBehave: Primarily uses Java.
    - Cucumber: Originally written in Ruby, but now supports multiple languages including Java, JavaScript, and more.
- **Configuration:**
    - JBehave: Requires more configuration and setup, particularly for specifying story paths and other settings.
    - Cucumber: Generally easier to set up with simpler configurations.
- **Community and Ecosystem:**
    - JBehave: Has a smaller community and fewer plugins and integrations compared to Cucumber.
    - Cucumber: Larger community, more plugins, and better integration with various tools and frameworks.
- **Annotations:**
    - JBehave: Uses Java methods annotated with JBehave-specific annotations like @Given, @When, and @Then.
    - Cucumber: Uses annotations similar to JUnit, like @Given, @When, and @Then.
- **Usage:**
    - JBehave: More commonly used in Java-centric projects where developers are comfortable with Java configuration.

- Cucumber: Widely used across different languages and frameworks due to its flexible support and simpler syntax

31. What do you understand by TDD, and what are the different processes used in TDD?

Test-Driven Development (TDD) is a software development approach where you write tests for your code before you write the code itself. The process helps ensure that the code is working as expected and helps in creating cleaner, more reliable code.

Processes used in TDD:

1. **Write a Test:** Write a test for a small piece of functionality that you want to add.
2. **Run the Test:** Run the test to see it fail. This step ensures that the test is valid and that the functionality does not yet exist.
3. **Write Code:** Write the minimal amount of code needed to make the test pass.
4. **Run the Test Again:** Run the test again to see it pass.
5. **Refactor:** Refactor the code to improve its structure and readability, while ensuring that all tests still pass.
6. **Repeat:** Repeat the cycle for the next piece of functionality.

32. What are the similarities between BDD and TDD?

Similarities between BDD and TDD:

1. **Test-First Approach:** Both BDD and TDD involve writing tests before writing the actual code.
2. **Automation:** Both methodologies promote automated testing to ensure code correctness and functionality.
3. **Frequent Testing:** They encourage frequent testing throughout the development process to catch defects early.
4. **Refactoring:** Both practices include refactoring code to improve design and maintainability without changing its behavior.
5. **Improved Code Quality:** Both aim to produce higher quality, more reliable code by ensuring that it meets defined requirements.

33. What are the main differences between TDD and BDD?

Differences between TDD and BDD:

1. **Focus:**

- **TDD:** Focuses on testing individual units of code (e.g., methods, functions) to ensure they work correctly.
- **BDD:** Focuses on the behavior of the application from the user's perspective, ensuring that the system meets business requirements.

2. **Language:**
    - **TDD:** Tests are written in the programming language of the application, using frameworks like JUnit, NUnit, etc.
    - **BDD:** Tests are written in a natural, human-readable language using frameworks like Cucumber, which uses Gherkin syntax.
3. **Scope:**
    - **TDD:** Typically tests small units of code in isolation.
    - **BDD:** Tests the behavior of the entire system or specific features, often involving multiple units of code and their interactions.
4. **Collaboration:**
    - **TDD:** Primarily involves developers.
    - **BDD:** Encourages collaboration between developers, testers, business analysts, and other stakeholders.
5. **Documentation:**
    - **TDD:** Produces technical documentation in the form of unit tests.
    - **BDD:** Produces readable documentation that can be understood by non-technical stakeholders.

41. What is a data table in Cucumber?

A **data table** in Cucumber is a way to pass a set of data to a step in a structured format. It allows you to provide multiple values for a single step, making it easier to test different scenarios without writing repetitive steps. Data tables are typically used to input data into forms, validate multiple fields, or verify lists.

Example:

Scenario: Verify user details
Given the following user details:

name	email	age
John	john@example.com	30
Jane	jane@example.com	25

Step Definition:

```
@Given("the following user details:")
public void userDetails(DataTable table) {
   List<Map<String, String>> userDetails = table.asMaps(String.class, String.class);
   for (Map<String, String> user : userDetails) {
      // Code to process user details
   }
}
```

43. How do you skip a scenario in Cucumber?

You can skip a scenario in Cucumber by using tags. By tagging a scenario with a specific tag ***(e.g., @skip)***, you can configure your test runner to exclude scenarios with that tag during execution.

Example:

```
@skip
Scenario: This scenario will be skipped
  Given some precondition
  When some action is taken
  Then some expected result is obtained
```

In your test runner, exclude the @skip tag:

cucumber --tags "not @skip"

44. How do you generate HTML reports in Cucumber?

To generate HTML reports in Cucumber, you need to use a reporting plugin.
The cucumber-html-reporter is a popular choice. You configure your test runner to generate the report.

Example in Java with Maven and Cucumber:

A. Add the required dependencies to your pom.xml:

```
<dependency>
  <groupId>io.cucumber</groupId>
  <artifactId>cucumber-java</artifactId>
  <version>6.10.4</version>
</dependency>
<dependency>
```

```
 <groupId>io.cucumber</groupId>
 <artifactId>cucumber-junit</artifactId>
 <version>6.10.4</version>
</dependency>
```

B. Configure the @CucumberOptions annotation in your test runner:

```
@RunWith(Cucumber.class)

@CucumberOptions(

  features = "src/test/resources/features",

  glue = "stepDefinitions",

  plugin = {"pretty", "html:target/cucumber-reports.html"}

)

public class TestRunner {

}
```

C. Run your tests. The HTML report will be generated in the **target/cucumber-reports.html** file.

51. What is a cucumber world in Cucumber?

In Cucumber, a **world** is a context object that holds shared state and data between different steps in a scenario. It allows you to maintain and access common information throughout the scenario execution. The world is typically used to store data that needs to be accessed by multiple step definitions, helping to avoid global variables and keep the code clean.

Example:

```
public class MyWorld {

  public String username;

  public String password;

}


// Step definitions can then use MyWorld to share state

@Given("the user has a username {string}")

public void setUsername(String username) {
```

```
    myWorld.username = username;
}


@When("the user logs in")
public void loginUser() {
    // Use myWorld.username for login
}
```

53. What is the difference between a scenario and a step in Cucumber?

Scenario:

- **Purpose:** A scenario is a specific test case that describes a particular use case or situation to be tested.
- **Structure:** Consists of multiple steps (Given, When, Then).
- **Scope:** Represents a complete test case.
- **Keyword:** Starts with the keyword Scenario.

Example:

Scenario: Successful login with valid credentials
Given the user is on the login page
When the user enters valid username and password
Then the user should be redirected to the homepage

Step:

- **Purpose:** A step is an individual action or check within a scenario.
- **Structure:** Represents a single action or assertion.
- **Scope:** Part of a scenario.
- **Keywords:** Starts with keywords like Given, When, Then, And, or But.

Example:

Given the user is on the login page
When the user enters valid username and password
Then the user should be redirected to the homepage

54. What is the difference between JUnit and Cucumber?

JUnit:

- **Purpose:** A unit testing framework for Java applications, primarily used for testing individual units of code (methods, classes).
- **Syntax:** Uses annotations like @Test, @Before, and @After.
- **Focus:** Tests isolated pieces of code.
- **Language:** Java.
- **Use Case:** Unit testing.

Example:

```
public class ExampleTest {
  @Test
  public void testAddition() {
    assertEquals(5, 2 + 3);
  }
}
```

Cucumber:

- **Purpose:** A behavior-driven development (BDD) framework used for writing acceptance tests in plain English.
- **Syntax:** Uses Gherkin language with keywords like Feature, Scenario, Given, When, Then.
- **Focus:** Tests the behavior of the entire application from the user's perspective.
- **Language:** Multiple languages (Java, Ruby, JavaScript, etc.).
- **Use Case:** Acceptance testing, end-to-end testing.

Example:

Feature: User Login

Scenario: Successful login with valid credentials
Given the user is on the login page
When the user enters valid username and password
Then the user should be redirected to the homepage

59. What is the difference between a soft assertion and a hard assertion in Cucumber?

Soft Assertion:

- **Behavior:** Continues executing the test even if the assertion fails, allowing multiple assertions to be checked within a single test.
- **Use Case:** Useful for validating multiple conditions in a test scenario and reporting all failures at once.
- **Example:** In Java, soft assertions can be implemented using libraries like softassert.

Example in Java:

SoftAssert softAssert = **new** SoftAssert();

softAssert.assertTrue(condition1, "Condition 1 failed");

softAssert.assertTrue(condition2, "Condition 2 failed");

softAssert.assertAll(); *// Reports all assertion failures*

Hard Assertion:

- **Behavior:** Stops test execution immediately if the assertion fails, preventing any further steps or assertions from being executed.
- **Use Case:** Useful for critical checks where subsequent steps depend on the success of the current assertion.
- **Example:** In Java, hard assertions can be implemented using Assert class from JUnit or TestNG.

Example in Java:

Assert.assertTrue(condition1, "Condition 1 failed");

Assert.assertTrue(condition2, "Condition 2 failed"); *// This will not be executed if condition1 fails*

60. How do you use Cucumber hooks?

Cucumber hooks are used to perform actions before or after scenarios, features, or steps. They help set up preconditions and clean up after tests. Commonly used hooks are @Before and @After.

1. **@Before Hook:**
    - Runs before each scenario to set up the initial context or state.
2. **@After Hook:**
    - Runs after each scenario to clean up or reset the state.

Example in Java:

```
import io.cucumber.java.Before;
import io.cucumber.java.After;


public class Hooks {
  @Before
  public void setUp() {
    // Code to set up preconditions, e.g., open a browser, initialize test data
    System.out.println("Setting up before scenario");
  }


  @After
  public void tearDown() {
    // Code to clean up after the scenario, e.g., close the browser, reset data
    System.out.println("Cleaning up after scenario");
  }
}
```

Example in Gherkin:

```
Feature: User Login

 Background:
  Given the database is clean

 @BeforeScenario
 Scenario: Successful login
  Given the user is on the login page
  When the user enters valid credentials
  Then the user should be redirected to the homepage

 @AfterScenario
 Scenario: Unsuccessful login
  Given the user is on the login page
  When the user enters invalid credentials
  Then the user should see an error message
```

In this example, the @Before hook sets up the initial conditions before each scenario, and the @After hook cleans up after each scenario

61. How do you use Cucumber with Selenium?

To use **Cucumber with Selenium**, you need to follow these steps:

A. Set Up Dependencies:

Add the required dependencies for Cucumber and Selenium in your build file (e.g., pom.xml for Maven or build.gradle for Gradle).

Example pom.xml:

```
<dependencies>
 <!-- Cucumber dependencies -->
 <dependency>
  <groupId>io.cucumber</groupId>
  <artifactId>cucumber-java</artifactId>
  <version>6.10.4</version>
 </dependency>
 <dependency>
  <groupId>io.cucumber</groupId>
  <artifactId>cucumber-junit</artifactId>
  <version>6.10.4</version>
 </dependency>

 <!-- Selenium dependency -->
 <dependency>
  <groupId>org.seleniumhq.selenium</groupId>
  <artifactId>selenium-java</artifactId>
  <version>3.141.59</version>
 </dependency>
</dependencies>
```

B. Create Feature Files:

Write feature files in Gherkin syntax to describe the behavior of your application.

Example Login.feature:

Feature: User Login

  Scenario: Successful login with valid credentials
    Given the user is on the login page

When the user enters valid username and password
Then the user should be redirected to the homepage

C. Write Step Definitions:

Implement step definitions in Java to map Gherkin steps to Selenium code.

Example LoginSteps.java:

```
import io.cucumber.java.en.Given;
import io.cucumber.java.en.When;
import io.cucumber.java.en.Then;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.By;
import org.openqa.selenium.WebElement;
import static org.junit.Assert.assertTrue;

public class LoginSteps {
  WebDriver driver;

  @Given("the user is on the login page")
  public void userOnLoginPage() {
    System.setProperty("webdriver.chrome.driver", "path/to/chromedriver");
    driver = new ChromeDriver();
    driver.get("http://example.com/login");
  }

  @When("the user enters valid username and password")
  public void userEntersCredentials() {
    WebElement username = driver.findElement(By.id("username"));
    WebElement password = driver.findElement(By.id("password"));
```

```
        WebElement loginButton = driver.findElement(By.id("loginButton"));
        username.sendKeys("validUser");
        password.sendKeys("validPassword");
        loginButton.click();
    }

    @Then("the user should be redirected to the homepage")
    public void userRedirectedToHomepage() {
        assertTrue(driver.getCurrentUrl().contains("homepage"));
        driver.quit();
    }
}
```

C. Run Tests:

Create a test runner class to run your Cucumber tests.

Example TestRunner.java:

```
import org.junit.runner.RunWith;
import io.cucumber.junit.Cucumber;
import io.cucumber.junit.CucumberOptions;

@RunWith(Cucumber.class)
@CucumberOptions(
    features = "src/test/resources/features",
    glue = "stepDefinitions",
    plugin = {"pretty", "html:target/cucumber-reports.html"}
)
public class TestRunner {
}
```

D. Execute Tests:

Run the test runner class to execute the Cucumber tests with Selenium.

Q. In methods names are same twice in step definition file what will happen? will it throw error? in cucumber

🧨 **Yes, Cucumber will throw an error!**

Cucumber will **fail at runtime** with an error like:

Code

cucumber.runtime.DuplicateStepDefinitionException: Duplicate step definitions in:

path.to.StepDefs.methodOne()

path.to.StepDefs.methodTwo()

API Testing

api what is ?
why we use api?
api test backend or frontend?
what are the things you will receive as response?
what is api chaining?

what are the request you will be sending

how you will retreive cookies and header?
what are types of authentication in api ?
which authenctication to prefer and why ?

Api response code

Status code and different types

What is endpoint?

Method to create request body –hashmap,pojo

✅ 1. What is an API?

API (Application Programming Interface) is a set of rules that allows two software systems to communicate with each other. It defines how requests and responses should be structured.

🧠 Think of it like a waiter in a restaurant — it takes your order (request), tells the kitchen (server), and brings back your food (response).

✅ 2. Why do we use APIs?

- 🔄 To enable communication between frontend and backend
- ⚙️ To reuse business logic across apps (e.g., login, payments)
- 📱 To connect mobile/web apps to servers
- 🤖 To automate workflows and integrate third-party services (e.g., payment gateways, weather APIs)

✅ 3. Is API testing frontend or backend?

API testing is backend testing.

- It validates the logic, data, and response of the server — without using the UI.
- You test endpoints like POST /login, GET /users, etc.
- Tools: Postman, RestAssured, SoapUI, JMeter

✅ 4. What do you receive in an API response?

A typical API response includes:

Component	Description
Status Code	HTTP code (e.g., 200 OK, 404 Not Found, 500 Error)
Headers	Metadata (e.g., content-type, auth tokens)
Body	Actual data (JSON, XML, HTML, etc.)
Time	Response time in milliseconds
Cookies	Session or tracking data (optional)

📜 Example JSON Response:

json

```
{
 "id": 101,
 "name": "Komal",
 "role": "QA Engineer",
 "status": "active"
}
```

🔗 What is API Chaining?

API Chaining is a technique where the output of one API request is used as input for the next request.

✅ Why use it?

- To simulate real-world workflows (e.g., login → create user → get user → delete user)
- To validate data consistency across endpoints
- To automate end-to-end backend flows

📜 Example:

1. **POST /login** → returns token
2. **POST /createUser** → uses token in header
3. **GET /getUser/{id}** → uses user ID from previous response

📤 What Types of Requests Will You Send?

Method	Purpose	Example Use Case
GET	Retrieve data	Get user details
POST	Create new resource	Register a new user
PUT	Update existing resource	Update user profile
PATCH	Partially update a resource	Update user email only

Method	Purpose	Example Use Case
DELETE	Remove a resource	Delete a user

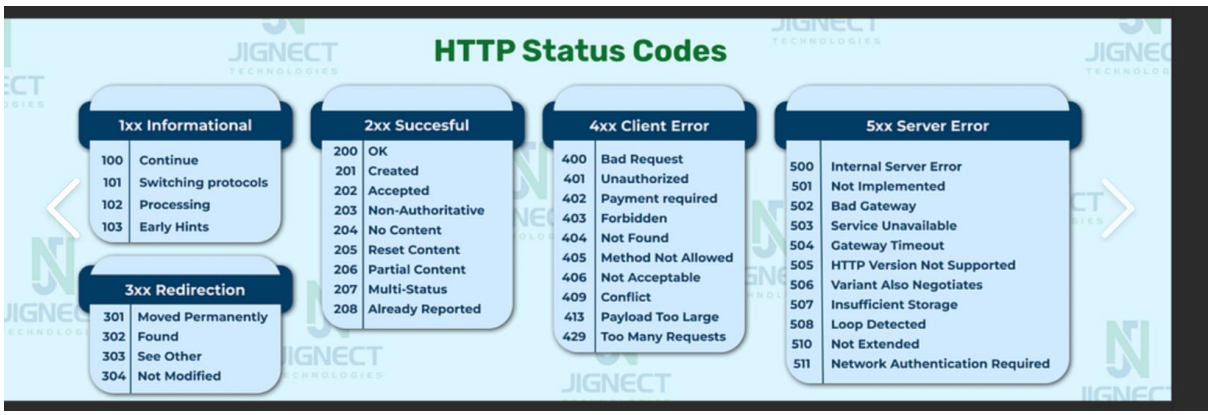
🌐 What is an API Response Code?

An **API response code** (also called **HTTP status code**) is a numeric code returned by the server to indicate the result of a client's request. It helps you understand whether the request was successful, failed, or needs further action.

✅ Common Status Codes and Their Types

Status codes are grouped into five categories:

Code Range	Type	Meaning & Examples
1xx	Informational	Request received, continuing process (rare in APIs)
2xx	Success	✅ Request was successful • 200 OK • 201 Created • 204 No Content
3xx	Redirection	🔁 Further action needed • 301 Moved Permanently • 302 Found
4xx	Client Error	❌ Problem with request • 400 Bad Request • 401 Unauthorized • 403 Forbidden • 404 Not Found
5xx	Server Error	⚠️ Server failed to fulfill request • 500 Internal Server Error • 502 Bad Gateway • 503 Service Unavailable



📍 What is an Endpoint?

An **API endpoint** is a specific URL where an API can be accessed by a client. It represents a resource or service.

Example:

Code

https://api.example.com/users/123

- https://api.example.com → Base URL
- /users/123 → Endpoint to get user with ID 123

In Selenium-based API testing, you often hit endpoints using tools like **RestAssured**, **Postman**, or **Java HTTP clients**.

📝 How to Create a Request Body

📜 1. Using HashMap

This is a quick and flexible way to build a dynamic request body.

✅ Example:

java

```
Map<String, Object> requestBody = new HashMap<>();
requestBody.put("name", "Komal");
requestBody.put("email", "komal@example.com");
requestBody.put("role", "QA Engineer");
```

🔧 Usage with RestAssured:

java

```
given()
  .contentType("application/json")
  .body(requestBody)
.when()
```

```
 .post("/createUser")
.then()
 .statusCode(201);
```

🔍 Pros:

- Easy for dynamic key-value pairs
- No need for extra classes

⚠️ Cons:

- No compile-time safety
- Harder to maintain for large payloads

🧱 2. Using POJO (Plain Old Java Object)

This is the cleanest and most scalable method, especially for structured data.

✅ POJO Class:

```
java
public class User {
  private String name;
  private String email;
  private String role;

  // Constructors, Getters, Setters
}
```

🔧 Usage:

```
java
User user = new User();
user.setName("Komal");
user.setEmail("komal@example.com");
user.setRole("QA Engineer");
```

```
given()
  .contentType("application/json")
  .body(user)
.when()
  .post("/createUser")
.then()
  .statusCode(201);
```

🔍 Pros:

- Clean and readable
- IDE support and compile-time safety
- Easy to reuse and test

⚠️ Cons:

- Requires extra class creation

📜 3. Using Raw JSON String

Useful for quick tests or when copying from documentation.

✅ Example:

java

```
String jsonBody = "{ \"name\": \"Komal\", \"email\": \"komal@example.com\", \"role\": \"QA Engineer\" }";
```

🔧 Usage:

java

```
given()
  .contentType("application/json")
  .body(jsonBody)
.when()
  .post("/createUser")
```

.then()

.statusCode(201);

🔍 Pros:

- Fast for prototyping
- Easy to copy-paste

⚠️ Cons:

- Error-prone (no validation)
- Hard to maintain

🧪 Interview Tip

- Prefer POJO for real-world frameworks.
- Use HashMap for dynamic or partial updates.
- Mention Jackson/Gson for serialization.
- Show how it integrates with RestAssured, TestNG, or Spring Boot.

🧪 Postman Test Scripts – Essentials

Postman test scripts are written in JavaScript under the Tests tab of a request. They allow you to:

Validate API responses

Extract and reuse data (chaining)

Set environment or global variables

Log output for debugging

✅ 1. Basic Status Code Check

javascript

```
pm.test("Status code is 200", function () {
    pm.response.to.have.status(200);
});
```

✅ 2. Validate Response Body

javascript

```
pm.test("Response has expected user ID", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData.id).to.equal(101);
});
```

✅ 3. Check Response Headers

javascript

```
pm.test("Content-Type is JSON", function () {
    pm.response.to.have.header("Content-Type");
    pm.expect(pm.response.headers.get("Content-Type")).to.include("application/json");
});
```

✅ 4. Validate Response Time

javascript

```
pm.test("Response time is under 500ms", function () {
    pm.expect(pm.response.responseTime).to.be.below(500);
});
```

✅ 5. Extract and Chain Data (Set Variable)

javascript

```
var jsonData = pm.response.json();
pm.environment.set("userToken", jsonData.token);
```

You can now use {{userToken}} in headers of the next request.

✅ 6. Validate Cookies

javascript

```
pm.test("Session cookie is present", function () {
  var cookie = pm.cookies.get("session_id");
  pm.expect(cookie).to.not.be.undefined;
});
```

✅ 7. Loop Through JSON Array

javascript

```
pm.test("All users have email field", function () {
  var jsonData = pm.response.json();
  jsonData.forEach(function(user) {
    pm.expect(user).to.have.property("email");
  });
});
```

Q. How you will retreive cookies and header?

🧪 **Using Postman**

🔹 **View Response Headers:**

1. Send the request.
2. Go to the **“Headers”** tab in the response section.
3. You’ll see all response headers like Content-Type, Authorization, Set-Cookie, etc.

🔹 **Retrieve Headers in Tests:**

javascript

```
pm.test("Check Content-Type", function () {
  pm.response.to.have.header("Content-Type");
});
```

```
let contentType = pm.response.headers.get("Content-Type");
console.log("Content-Type:", contentType);
```

🔹 **Retrieve Cookies:**

```
javascript
let sessionCookie = pm.cookies.get("JSESSIONID");
console.log("Session ID:", sessionCookie);
```

☕ **Using Rest Assured (Java)**

🔹 **Get Headers:**

```
java
Response response = given()
   .when()
   .get("/api/endpoint");

String contentType = response.getHeader("Content-Type");
Headers allHeaders = response.getHeaders();
System.out.println("Content-Type: " + contentType);
```

🔹 **Get Cookies:**

```
java
String sessionId = response.getCookie("JSESSIONID");
Map<String, String> allCookies = response.getCookies();
System.out.println("Session ID: " + sessionId);
```

You can also use:

```
java
Cookie cookie = response.getDetailedCookie("JSESSIONID");
System.out.println("Cookie Path: " + cookie.getPath());
```

🔹 **OPTIONS Request**

✅ What is it?

The **OPTIONS** method is used to describe the communication options available for a specific resource or server.

🧠 Why is it used?

- To check which HTTP methods (GET, POST, PUT, DELETE, etc.) are supported by the server for a given endpoint
- Commonly used in **CORS (Cross-Origin Resource Sharing)** preflight checks

📜 Example in Postman:

- Method: OPTIONS
- URL: https://api.example.com/users
- Response headers might include:

Code

Allow: GET, POST, OPTIONS

Access-Control-Allow-Methods: GET, POST, OPTIONS

🔹 HEAD Request

✅ What is it?

The **HEAD** method is similar to GET, but it only retrieves the **headers** — not the response body.

🧠 Why is it used?

- To check if a resource exists without downloading the entire content
- To verify metadata like content type, length, or last modified date
- Useful for performance checks or caching strategies

📜 Example in Postman:

- Method: HEAD
- URL: https://api.example.com/users/123
- Response:
    - Status: 200 OK
    - Headers:

Code

💬 **Interview-Ready Summary**

Q: What's the difference between OPTIONS and HEAD requests? A: "OPTIONS is used to discover which HTTP methods are allowed on a resource — often for CORS validation. HEAD is like GET but returns only headers, which helps check resource availability or metadata without downloading the full response."

💉 **Data-Driven Testing in Postman Using CSV**

✅ **1. Prepare Your CSV File**

Create a CSV file with column headers and test data:

📄 LoginData.csv

csv

username,password

komal,pass123

admin,admin@123

testuser,test@456

✅ **2. Use Variables in Your Request**

In your request body or URL, use double curly braces to reference CSV columns:

📄 Request Body (JSON):

json

{

"username": "{{username}}",

"password": "{{password}}"

}

📄 URL Example:

Code

https://api.example.com/login?user={{username}}&pass={{password}}

✅ **3. Run Collection with CSV Data**

1. Click **Runner** in Postman

2. Select your **collection**
3. Click **Data** → Upload LoginData.csv
4. Click **Run**

Postman will execute the request once for each row in the CSV file.

```
package com.studentapp.tests;

import org.junit.jupiter.api.BeforeAll;

import io.restassured.RestAssured;

public class TestBase {


   @BeforeAll
   public static void init() {

      RestAssured.baseURI = "http://localhost";
      RestAssured.port = 8080;
      RestAssured.basePath = "/student";
   }

}

package com.studentapp.tests;

import org.junit.jupiter.api.Disabled;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import io.restassured.RestAssured;
import io.restassured.response.Response;
import io.restassured.response.ValidatableResponse;
import io.restassured.specification.RequestSpecification;

import static io.restassured.RestAssured.*;

import java.util.HashMap;
import java.util.Map;

public class StudentGetRequestTests extends TestBase{

   private void styles(){

      RestAssured.given()
```

```
            .queryParam("", "")
            .when()
            .get("https://www.google.com/")
            .then();


    RestAssured.given()
            .expect()
            .when();

}


@DisplayName("Getting all the student from the database")
@Test
void getAllStudents() {
    /*
    RequestSpecification requestSpec =  RestAssured.given();

    Response response = requestSpec.get("http://localhost:8080/student/list");

    response.prettyPrint();


    ValidatableResponse validatableResponse = response.then();

    validatableResponse.statusCode(200); */

    RestAssured.given()
            .when()
            .get("/list")
            .then()
            .statusCode(200);

    RestAssured.given()
            .expect()
            .statusCode(200)
            .when()
            .get("/list");

    given()
      .when()
      .get("/list")
      .then()
      .statusCode(200);

}
```

```
  //@Disabled
  @DisplayName("Get a single CS student from the list")
  @Test
  void getSingleCSStudent() {

     Map<String,Object> params =  new HashMap<String, Object>();
     params.put("programme", "Computer Science");
     params.put("limit",1);


     Response response =
     given()
     //.queryParam("programme", "Computer Science")
     //.queryParam("limit", 1)
     //.queryParams("programme", "Computer Science", "limit", 1)
     .queryParams(params)
     .when()
     .get("/list");

     response.prettyPrint();

  }

  @DisplayName("PathParameterExample: Get the firstStudent")
  @Test
  void getTheFirstStudent() {

     Response response =
           given()
           .pathParam("id", 2)
           .when()
           .get("/{id}");

     response.prettyPrint();

//     RestAssured.reset();
//
//     Response response2 =
//           given()
//           .pathParam("id", 2)
//           .when()
//           .get("/{id}");
//
//     response.prettyPrint();

  }
```

```
}

package com.studentapp.tests;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import com.github.javafaker.Faker;
import com.studentapp.model.StudentPojo;

import io.restassured.http.ContentType;

import static io.restassured.RestAssured.*;

import java.util.ArrayList;
import java.util.List;

public class UpdateStudentEmailTest extends TestBase {

   @DisplayName("Update student email")
   @Test
   void updateStudentEmail() {

      StudentPojo student = new StudentPojo();
      Faker fake = new Faker();


      student.setEmail(fake.internet().emailAddress());


      given()
      .when()
      .contentType(ContentType.JSON)
      .when()
      .body(student)
      .patch("/101")
      .then()
      .statusCode(200);
   }
}

package com.studentapp.tests;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
```

```
import com.github.javafaker.Faker;
import com.studentapp.model.StudentPojo;

import io.restassured.http.ContentType;

import static io.restassured.RestAssured.*;

import java.util.ArrayList;
import java.util.List;

public class UpdateStudentPojoPayloadTest extends TestBase {

   @DisplayName("Update a new student by sending payload as an object")
   @Test
   void updateStudent() {

      StudentPojo student = new StudentPojo();
      Faker fake = new Faker();

      List<String> courses = new ArrayList<String>();
      courses.add("Java");
      courses.add("C++");

      student.setFirstName(fake.name().firstName());
      student.setLastName(fake.name().lastName());
      student.setEmail(fake.internet().emailAddress());

      student.setProgramme("Computer Science");
      student.setCourses(courses);

      given()
      .when()
      .contentType(ContentType.JSON)
      .when()
      .body(student)
      .put("/101")
      .then()
      .statusCode(200);
   }
}

package com.studentapp.tests;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import com.github.javafaker.Faker;
```

```
import com.studentapp.model.StudentPojo;

import io.restassured.http.ContentType;

import static io.restassured.RestAssured.*;

import java.util.ArrayList;
import java.util.List;

public class DeleteStudentTest extends TestBase {

   @DisplayName("Delete Student from the system")
   @Test
   void deleteStudent() {


      given()
      .when()
      .contentType(ContentType.JSON)
      .when()
      .delete("/101")
      .then()
      .statusCode(204);

   }
}
```

```
given().get("/users").then().statusCode(200);
```

✅ **3. Query & Path Parameters**

java

```
given().queryParam("page", 2)

.when().get("/users")

.then().statusCode(200);


given().pathParam("id", 101)

.when().get("/users/{id}")
```

.then().statusCode(200);

✅ **4. Request Body (String, POJO, Map)**

java

// As String

given().body("{\"name\":\"Komal\"}")

// As Map

Map<String, String> data = new HashMap<>();

data.put("name", "Komal");

given().body(data)

// As POJO

User user = new User("Komal", "QA");

given().body(user)

🧠 **1. What Is JsonPath?**

- JsonPath is like XPath but for JSON.
- It lets you **navigate**, **filter**, and **extract** data from JSON documents using path expressions.
- Jayway JsonPath is the **Java implementation** widely used in test automation, especially with tools like RestAssured, Selenium, and TestNG.

🧩 **2. Core Syntax**

Symbol	Meaning
$	Root of the JSON document
.	Child operator
..	Recursive descent

Symbol	Meaning
*	Wildcard (matches all elements)
[]	Subscript operator (index or filter)
[?()]	Filter expression
@	Refers to current element in filter

📚 3. Examples

Given this JSON:

```
json
{
 "store": {
  "book": [
   { "title": "Java", "price": 10 },
   { "title": "Selenium", "price": 15 }
  ],
  "bicycle": { "color": "red", "price": 19.95 }
 }
}
```

JsonPath	Result
$.store.book[*].title	All book titles
$..price	All prices
$.store.book[0]	First book
$.store.book[?(@.price < 12)]	Books cheaper than 12

🧪 4. Filter Expressions

- Use ?() inside brackets to filter arrays.
- Example: $.store.book[?(@.title == 'Java')]
- Supports operators: ==, !=, <, >, <=, >=, =~ (regex)

🛠️ 5. Functions

Function	Purpose
length()	Returns array length
min(), max()	Min/max value in array
avg()	Average value
sum()	Sum of values
keys()	Returns keys of an object

Example: $.store.book.length() → returns 2

📝 7. Integration in Java

java

```
String json = "{ \"name\": \"Komal\", \"skills\": [\"Selenium\", \"TestNG\"] }";

String skill = JsonPath.read(json, "$.skills[0]");

System.out.println(skill); // Selenium
```

```
package com.bestbuy.response.extraction;
import static io.restassured.RestAssured.given;

import java.util.HashMap;
import java.util.List;
import java.util.Map;

import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeAll;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import com.jayway.jsonpath.JsonPath;

import io.restassured.RestAssured;

public class JsonPathJaywayExamples {


    static String jsonResponse;
```

```
 static void print(String val){
    System.out.println("----------------------------------");
    System.out.println(val);
    System.out.println("----------------------------------");
 }

@BeforeAll
 public static void initialize(){

    RestAssured.baseURI = "http://localhost";
    RestAssured.port = 3030;

    jsonResponse = given().when().get("/products").asString();

 }

@BeforeEach
void printToConsole() {
   System.out.println("-----Starting the test method--------");
   System.out.println("   ");
}

@AfterEach
void printToConsoleAgain() {
   System.out.println("-----Ending the test method--------");
   System.out.println("   ");
}

 @DisplayName("Get the root element")
 @Test
 public void getRoot(){

    Map<String,?> rootElement = JsonPath.read(jsonResponse, "$");
    print(rootElement.toString());
 }


 @DisplayName("Get the total value from the response")
 @Test
 public void getTotalFromResponse(){

    int totalValue = JsonPath.read(jsonResponse, "$.total");
    print(totalValue + "");

 }
```

```
    @DisplayName("Get all the data elements")
    @Test
    public void getAllDataElements(){

        List<HashMap<String,Object>>  dataElements= JsonPath.read(jsonResponse,
"$.data");
        dataElements.stream().forEach(System.out::println);
    }


    @DisplayName("Get firstDataElement")
    @Test
    public void getFirstDataElement(){

        Map<String,?> firstDataElement = JsonPath.read(jsonResponse, "$.data[0]");
        print(firstDataElement.toString());

    }


    @DisplayName("Get lastDataElement")
    @Test
    public void getLastDataElement(){

        Map<String,?> lastDataElement = JsonPath.read(jsonResponse, "$.data[-1]");
        print(lastDataElement.toString());

    }


    @DisplayName("Get all the ids in data")
    @Test
    public void getAllIdsUnderData(){

        List<String> lastDataElement = JsonPath.read(jsonResponse, "$.data[*].id");
        print(lastDataElement.toString());

    }


    @DisplayName("Get All the Ids")
    @Test
    public void getAllIds(){

        List<String> allIds = JsonPath.read(jsonResponse, "$..[*].id");
        print(allIds.toString());
```

```
    }

    @DisplayName("Get The name of the products whose price is less than 5")
    @Test
    public void getNameOfProductsWhosePriceisLessThan5(){

        List<String> names = JsonPath.read(jsonResponse,
"$.data[?(@.price>5)].name");

        names.stream().forEach(System.out::println);
    }

}
```

Docker Commands

Basic
****
1)docker version
2)docker -v
3)docker info
4)docker --help

docker --help
Example: Get information about images.....
docker images --help --> Gives you details about how to use images command
docker run --help --> Gives you details about how to use run commands
5)docker login

Images
****
6)docker images --> lists the images present in the machine
7)docker pull ---> pull the image from docker hub
docker pull ubuntu
docker images --> list the image

8)docker rmi

docker images -q --> Gives you image ID

docker rmi <<image ID>> ----> Deletes image

docker images ---> No Images

Containers

****

9)docker ps & : docker run

docker ps ---> List the containers, Bo containers as of now.

docker run ubuntu ---> Locally not available ,Pulling image from Dockerhub.

docker ps --> Still not listed contaner, becoz we didi not created container so far....

docker run -it ubuntu ---> inside ubuntu

docker ps ---> lists the container

10) docker start

docker start <<container id>>

11)docker stop

docker stop <<container id>>

System

****

12) docker stats --> gives details about running containers, memory usage etc.

13)docker system df

14)docker system prune

docker system prune -f --> Removes all stopped containers